Chapter 3 Processing and Understanding Text

Correct Spellings

Main Code Components by Peter Norvig from http://norvig.com/spell-correct.html

Main Objective:
Given a problematic word, find most likely correct form of that word.
Also, illustrate "distance".

Approach:

- Generate a set of candidate words that are near to input word
- Select the most likely word from this set as correct word
- Use a corpus of correct English words to identify correct word based on its frequency in corpus
- Candidates are determined by edit distance to input word

Load Vocabulary from Corpus

```
import re
```

```
from collections import Counter
```

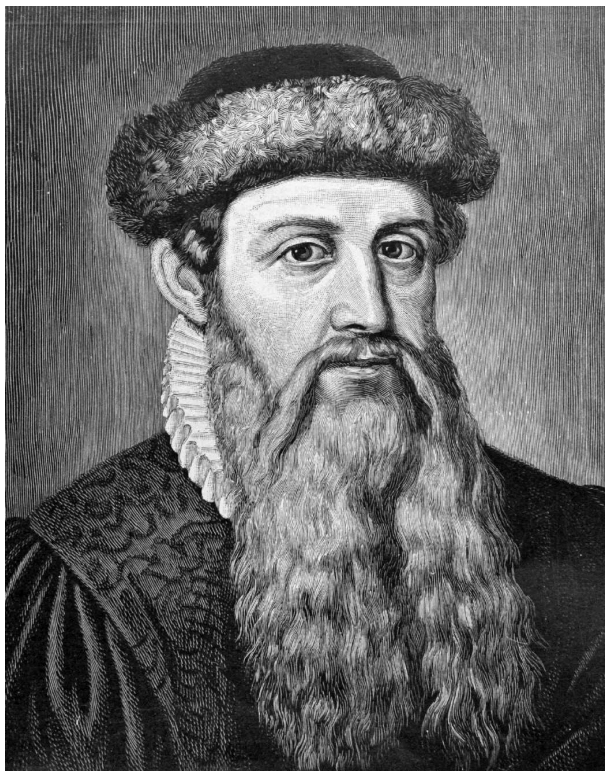
```
def words(text): return re.findall(r'\w+', text.lower())
```

```
words(open('big.txt').read())
```

```
Counter(words(open('big.txt').read()))
```

```
WORDS = Counter(words(open('big.txt').read()))
```

Johannes Gutenberg



https://en.wikipedia.org/wiki/Johannes_Gutenberg

Probability of a Word

Based on a particular corpus

```
def P(word, N=sum(WORDS.values())):
    "Probability of 'word'."
    return WORDS[word] / N
```

```
P('j0hn')
```

```
P('john')
```

Known Words

- Function `known()` returns subset of words from our candidate set
- Candidate set is produced by `edit` functions (see next slides)
- Do those words occur in the vocabulary dictionary?
- So, we get list of valid words from set of candidate words

```
def known(words):
    "The subset of 'words' that appear in the dictionary of WORDS."
    return set(w for w in words if w in WORDS)
```

```
known(["john", "watson", "abcd"])
```

Candidates through Edits

- Compute sets of words that are one or two edits away from input word
- Edits can be made through:
    - deletion (remove one letter)
    - transposition (swap two adjacent letters)
    - replacement (change one letter to another)
    - insertion (add a letter)

For word of length n, there will be

- n deletions
- n-1 transpositions
- 26n alterations
- 26(n+1) insertions

for a total of 54n+25 (of which a few are typically **duplicates**).

Edits 1

```
def edits1(word):
    "All edits that are one edit away from 'word'."
    letters = 'abcdefghijklmnopqrstuvwxyz'
    splits = [(word[:i], word[i:]) for i in range(len(word) + 1)]
    deletes = [L + R[1:] for L,R in splits if R]
    transposes = [L + R[1] + R[0] + R[2:] for L,R in splits if len(R) > 1]
    replaces = [L + c + R[1:] for L,R in splits if R for c in letters]
    inserts = [L + c + R for L,R in splits for c in letters]
    return set(deletes + transposes + replaces + inserts)
```

```
edits1("watson")
```

```
len(edits1("watson"))
# Using the formula above at most 54*6 + 25 = 349.
```

Edits 2

```
def edits2(word):
    "All edits that are two edits away from 'word'."
    return (e2 for e1 in edits1(word) for e2 in edits1(e1))
```

```
edits2words = edits2("watson")
```

```
edits2words
```

```
c = 0
for w in edits2words:
    print(c, w)
    c += 1
```

Candidates

```
def candidates(word):
    "Generate possible spelling correction for 'word' in a SET."
    return (known([word]) or known(edits1(word))
            or known(edits2(word)) or {word})
```

```
candidates("ture")
```

```
P("cure")
```

```
P("tyre")
```

```
P("true")
```

Correction

```
def correction(word):
    "Most probable spelling correction for word."
    return max(candidates(word), key=P)
```

```
correction("ture")
```

```
# correction will not correct unknown words,
# for which no known edits are found:
correction('magnifique')
```

Additional Readings

https://norvig.com/spell-correct.html

Text Analytics with Python: A Practitioner's Guide to Natural Language Processing
Paperback, 2019, by Dipanjan Sarkar